SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I R.Yaswanth (192312408) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

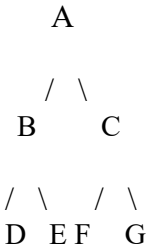
Signature of the student: R.Yaswanth

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



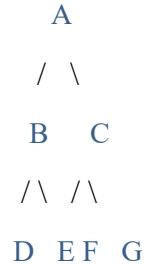
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached. Fill in

the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Question 1: Breadth-First Search (BFS)

Graph: A is the root; B and C are its children; D, E are children of B; F, G are children of C. BFS uses a **FIFO queue** and visits nodes level by level, adding children left to right. In each iteration the front node is removed and visited, and its children are added to the rear. The queue column shows the queue **after** that expansion (the queue starts as [A]).

Iteration	Node removed / expanded	Queue (after expansion)	Visited nodes so far
1	A	B, C	A
2	B	C, D, E	A, B
3	C	D, E, F, G	A, B, C
4	D	E, F, G	A, B, C, D
5	E	F, G	A, B, C, D, E
6	F	G	A, B, C, D, E, F
7	G	(empty)	A, B, C, D, E, F, G

1. BFS traversal order

A - B - C - D - E - F - G

2. Queue contents after each iteration

[B, C] - [C, D, E] - [D, E, F, G] - [E, F, G] - [F, G] - [G] - [ ] (empty). The queue is empty after iteration 7, so the search ends with all 7 nodes visited.

Question 2: Depth-First Search (DFS)

DFS uses a **LIFO stack** and goes as deep as possible before backtracking. In each iteration the top node is popped and visited, and its children are pushed so that the **left child is on top** (right child pushed first). The stack is written bottom to top (the rightmost item is the top). The stack starts as [A].

Iteration	Node popped / visited	Stack (after expansion, top at right)	Visited nodes so far
1	A	C, B	A
2	B	C, E, D	A, B
3	D (leaf)	C, E	A, B, D
4	E (leaf)	C	A, B, D, E
5	C	G, F	A, B, D, E, C
6	F (leaf)	G	A, B, D, E, C, F
7	G (leaf)	(empty)	A, B, D, E, C, F, G

DFS traversal order: A - B - D - E - C - F - G. (The order differs from BFS because DFS finishes the whole left subtree of B before visiting C.)

Question 3: BFS on the 8-Puzzle

Initial state and **goal state** (_ = blank, shown as 0 in the problem):

Initial state	Goal state
1 3 6 5 _ 2 4 7 8	1 2 3 4 5 6 7 8 _

Method: BFS with a FIFO queue. Successors are generated by moving the blank in the fixed order **Up, Down, Left, Right**; a state already generated is not added again (repeated-state check). States are written row by row as 136/502/478, with 0 as the blank.

Table (first 5 iterations)

Iter	Current state (expanded)	Queue before expansion	Queue after expansion
1	136/502/478	[136/502/478]	[106/532/478, 136/572/408, 136/052/478, 136/520/478]
2	106/532/478	[106/532/478, 136/572/408, 136/052/478, 136/520/478]	[136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478]
3	136/572/408	[136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478]	[136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480]
4	136/052/478	[136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480]	[136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078]
5	136/520/478	[136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078]	[016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470]

Children generated: iteration 1 gives Up (106/532/478), Down (136/572/408), Left (136/052/478), Right (136/520/478); iteration 2 gives Left and Right only (Down would return to the initial state); iteration 3 gives Left and Right; iteration 4 gives Up and Down; iteration 5 gives Up and Down. The goal is at depth 8, so the table cannot reach it within 5 iterations; the search continues in the same way and the goal is found at the depth-8 level (see the path below).

1. Which node is expanded in each iteration?

Iteration 1: 136/502/478 (initial); 2: 106/532/478; 3: 136/572/408; 4: 136/052/478; 5: 136/520/478. After the root, iterations 2 to 5 expand the root's four children in the order they were generated (Up, Down, Left, Right), because BFS always takes the node at the front of the queue.

2. How many states are generated in total?

- After iteration 5: 1 initial + 4 + 2 + 2 + 2 + 2 = **13 states** generated (5 expanded, 8 in the queue).
- For the complete search (goal removed from the queue at iteration 311 with this move order): **500 states generated** including the initial state (499 new states), of which 310 were expanded before the goal and 189 remain in the queue. The exact count depends on the move order and on when the goal test is applied (on generation or on expansion).

3. Complete solution path

The shortest solution has **8 moves** (blank moves: Right, Up, Left, Down, Left, Down, Right, Right):

Step	Move	State	Step	Move	State
0	(start)	1 3 6 5 _ 2 4 7 8	5	Left	1 2 3 _ 5 6 4 7 8
1	Right	1 3 6 5 2 4 7 8	6	Down	1 2 3 4 5 6 _ 7 8
2	Up	1 3 _ 5 2 6 4 7 8	7	Right	1 2 3 4 5 6 7 _ 8
3	Left	1 _ 3 5 2 6 4 7 8	8	Right	1 2 3 4 5 6 7 8 _
4	Down	1 2 3 5 _ 6 4 7 8			

Path: 136/502/478 - 136/520/478 - 130/526/478 - 103/526/478 - 123/506/478 - 123/056/478 - 123/456/078 - 123/456/708 - 123/456/780 (goal).

4. Why BFS always finds the shortest solution in an unweighted 8-puzzle

- BFS expands nodes in order of depth: all states at depth d are expanded before any state at depth d+1, because the FIFO queue keeps shallower nodes in front.
- Every move in the 8-puzzle has the same cost (1), so path cost equals depth, and the shallowest goal is the cheapest goal.
- Hence the first time the goal is reached, no shorter path can exist (any shorter path would have been found at an earlier level). BFS is also complete, since the branching factor is finite (at most 4).
- Trade-off: it needs a lot of memory and time ($O(b^d)$), as seen from the 500 states generated even for an 8-move solution.